

Princípios de Projeto e SOLID

Objetivo do arquivo

Resumo dos principais princípios de projeto usados em POO para deixar o sistema mais flexível, reutilizável, coeso e com menor acoplamento.

O foco é revisar os conceitos que ajudam a entender as questões de prova e a relação entre princípios e padrões de projeto.

Conceitos fundamentais

Coesão

- Definição: grau em que os elementos de uma classe ou módulo pertencem ao mesmo assunto.
- Por que é importante: aumenta clareza, facilita manutenção e reduz mistura de responsabilidades.
- Exemplo simples: `Pedido` cuidar do cálculo do pedido, e não de e-mail, banco e relatório ao mesmo tempo.

Acoplamento

- Definição: grau de dependência entre partes do sistema.
- Por que é importante: quanto menor o acoplamento, mais fácil trocar, testar e evoluir partes do código.
- Exemplo simples: `PedidoService` depender de `NotificacaoService`, e não de `EmailService` direto.

Encapsulamento

- Definição: esconder detalhes internos e controlar o acesso ao estado e ao comportamento da classe.
- Por que é importante: protege regras internas e evita uso incorreto de atributos e operações.
- Exemplo simples: atributo `saldo` privado, acessado por métodos como `depositar()` e `consultarSaldo()`.

Abstração

- Definição: representar o essencial de um conceito, sem expor detalhes desnecessários.
- Por que é importante: simplifica o uso de objetos e ajuda a trabalhar com contratos mais estáveis.
- Exemplo simples: usar a interface `Notificacao` sem depender do detalhe de `EmailNotificacao` ou `SmsNotificacao`.

Herança

- Definição: mecanismo em que uma classe reaproveita atributos e métodos de outra.
- Por que é importante: pode facilitar reuso, mas deve ser usada com cuidado.
- Exemplo simples: `Funcionario` como classe base e `FuncionarioCLT` como subclasse.

Polimorfismo

- Definição: capacidade de tratar objetos diferentes por meio de uma mesma interface ou tipo comum.
- Por que é importante: permite substituir implementações sem mudar o código cliente.
- Exemplo simples: uma variável do tipo `DescontoStrategy` receber `DescontoVip` ou `DescontoPremium`.

Composição

- Definição: montar objetos maiores a partir de outros objetos.
- Por que é importante: costuma dar mais flexibilidade que herança e reduz dependência da hierarquia de classes.
- Exemplo simples: um `Cafe` receber adicionais como `Leite` e `Chantilly` por composição.

Delegação

- Definição: repassar uma tarefa para outro objeto fazer.
- Por que é importante: separa responsabilidades e ajuda a reutilizar comportamento.
- Exemplo simples: `Facade` ou `Decorator` encaminhar a chamada para um objeto interno.

Reuso

- Definição: reaproveitamento de código, estruturas ou comportamentos já existentes.
- Por que é importante: evita duplicação e acelera evolução do sistema.
- Exemplo simples: usar uma classe base, uma composição ou um padrão como `Strategy` para reaproveitar o que já existe.

Manutenibilidade

- Definição: facilidade de entender, corrigir e alterar o sistema.
- Por que é importante: um código mais mantível reduz custo de mudança e risco de erro.
- Exemplo simples: separar `PedidoRepository`, `EmailService` e `NotaFiscalService` em vez de colocar tudo em `Pedido`.

Extensibilidade

- Definição: facilidade de adicionar novos comportamentos com pouco impacto no código existente.
- Por que é importante: o sistema evolui sem exigir alterações grandes e frequentes.
- Exemplo simples: adicionar `DescontoBlack` criando uma nova classe, sem alterar a calculadora.

Baixo acoplamento

- Definição: dependência fraca entre classes ou módulos.
- Por que é importante: facilita troca de implementação, testes e evolução do sistema.
- Exemplo simples: depender de `ClienteRepository` ou `Notificacao` em vez de uma classe concreta.

Alta coesão

- Definição: classe ou módulo com responsabilidades relacionadas entre si.
- Por que é importante: deixa o código mais claro, previsível e fácil de manter.
- Exemplo simples: uma classe de e-mail cuidar só de envio de e-mail.

Separação de responsabilidades

- Definição: dividir o sistema em partes com funções bem definidas.
- Por que é importante: reduz classes grandes, melhora a organização e evita mistura de tarefas.
- Exemplo simples: `Pedido`, `PedidoRepository`, `EmailService` e `NotaFiscalService` separados.

Inversão de dependência

- Definição: módulos de alto nível e baixo nível devem depender de abstrações.
- Por que é importante: reduz acoplamento e permite trocar detalhes técnicos sem alterar regra de negócio.
- Exemplo simples: `PedidoService` receber `NotificacaoService` no construtor.

Programar para interfaces, não para implementações

- Definição: fazer o código depender de contratos, não de classes concretas.
- Por que é importante: aumenta flexibilidade e facilita substituição de componentes.
- Exemplo simples: trabalhar com `interface Notificacao` em vez de instanciar `EmailNotificacao` diretamente.

SOLID

SOLID é um conjunto de cinco princípios de projeto orientado a objetos.

Ele é usado para criar sistemas mais organizados, flexíveis, fáceis de manter, mais testáveis e com menos acoplamento.

S — Single Responsibility Principle

- Nome em português: Princípio da Responsabilidade Única.
- Ideia central: uma classe deve ter apenas um motivo para mudar.
- Problema que evita: classe com várias tarefas misturadas.
- Exemplo simples: separar `Pedido`, `PedidoRepository` e `EmailService`.
- Relação com padrões de projeto: `Facade`, `Decorator`, `Proxy` e `Observer` podem ajudar a separar responsabilidades; `Facade` deve tomar cuidado para não virar uma classe que faz tudo.

O — Open/Closed Principle

- Nome em português: Princípio Aberto/Fechado.
- Ideia central: classes devem estar abertas para extensão e fechadas para modificação.
- Problema que evita: alterações constantes em código já pronto, normalmente por causa de muitos `if` e `switch`.
- Exemplo simples: adicionar novos descontos criando novas classes de estratégia.
- Relação com padrões de projeto: `Strategy`, `Decorator`, `Factory`, `Observer`, `Adapter`, `Proxy`, `Template Method` e `Visitor` costumam ajudar a estender comportamento sem mexer na classe principal.

L — Liskov Substitution Principle

- Nome em português: Princípio da Substituição de Liskov.
- Ideia central: uma subclasse deve poder substituir a superclasse sem quebrar o sistema.
- Problema que evita: herança em que a subclasse não respeita o contrato esperado.
- Exemplo simples: se o código espera um **Pagamento**, qualquer implementação derivada deve funcionar corretamente como **Pagamento**.
- Relação com padrões de projeto: aparece quando há hierarquias de classes bem definidas; também ajuda a avaliar se herança, **Template Method** e **Visitor** estão coerentes.

I — Interface Segregation Principle

- Nome em português: Princípio da Segregação de Interfaces.
- Ideia central: interfaces devem ser pequenas e específicas.
- Problema que evita: classes obrigadas a implementar métodos que não usam.
- Exemplo simples: separar uma interface grande em **Trabalhador**, **Nadador** e **Comedor**, em vez de uma única interface genérica.
- Relação com padrões de projeto: **Adapter**, **Facade** e **Observer** costumam trabalhar melhor quando os contratos são pequenos e focados.

D — Dependency Inversion Principle

- Nome em português: Princípio da Inversão de Dependência.
- Ideia central: dependa de abstrações, não de classes concretas.
- Problema que evita: alto acoplamento com detalhes técnicos.
- Exemplo simples: **PedidoService** receber **NotificacaoService** em vez de criar **EmailService** internamente.
- Relação com padrões de projeto: **Factory**, **Strategy**, **Adapter**, **Proxy**, **Facade**, **Decorator** e **Observer** ajudam muito a aplicar esse princípio.

Tabela-resumo do SOLID

Princípio	Ideia principal	Problema que reduz	Exemplo de aplicação
S	Uma classe, um motivo para mudar	Mistura de responsabilidades	Separar Pedido , PedidoRepository e EmailService
O	Aberto para extensão, fechado para modificação	Alterações frequentes em código pronto	Criar nova Strategy para novo desconto
L	Subclasse deve substituir a superclasse	Herança quebrando contrato	Subtipos que respeitam o comportamento esperado
I	Interfaces pequenas e específicas	Métodos desnecessários em classes	Dividir interfaces grandes em contratos menores
D	Depender de abstrações	Alto acoplamento com classes concretas	PedidoService depender de NotificacaoService